Tetris - Reinforcement Learning[2]

Sebastian Fellner, Simon Speth, Benedikt Wiberg, Paul Kehnel

Motivation

Why reinforcement learning with Tetris?
Deepmind has shown in several papers that it is possible to use the same Deep Q-Network (DQN) to solve different atari 2600 games on a superhuman level[1].
It would be interesting to see if it is possible to transfer these techniques to a game that requires deeper understanding of the game mechanics. Because of the great variance of possible stones and the resulting task of planning Tetris is an NP-Hard problem[2].
We set our task to use Reinforcement Learning (RL) with Convolutional Neural Networks (CNN) to learn optimal control policies for Tetris.

[1]Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." arXiv preprint arXiv:1312.5602 (2013).
[2]Erik D. Demaine and Susan Hohenberger and David Liben-Nowell, Tetris is Hard, Even to Approximate,CoRR 2002.

Related Work

- **Playing Atari with Deep Reinforcement Learning:**[3] This paper by Deepmind is one of the most impressive works on Reinforcement Learning and can be seen as fundamental work for further improvements.
- **Rainbow: Combining Improvements in Deep Reinforcement Learning:**[4] This is the current state of the art, by combining different enhancements for DQN. This is crucial for selecting well fitting enhancements for our problem.

[3]Mnih, Volodymyr, et al. "Playing atari with deep reinforcement learning." arXiv preprint arXiv:1312.5602 (2013).
[4]Hessel, Matteo, et al. "Rainbow: Combining Improvements in Deep Reinforcement Learning." arXiv preprint arXiv:1710.02298 (2017).

Mathematical Background

Bellman equation: Used to derive a recursive decomposition of the optimal q-value function. Q^* returns for the state s and the action a the expected discounted future reward.
$Q^*(s,a) = \mathbb{E}_{s' \sim \mathcal{E}}[r + \gamma \max_{a'} Q^*(s', a')|s, a]$
Loss Function: We use the same loss function as presented in the Double DQN Paper from Hado van Hasselt.

$$\textbf{Reward Function} \quad r(x) = \begin{cases} -1 & \text{if dead,} \\ n/(h+1) & \text{if line cleared,} \\ 0.01 & \text{else} \end{cases}$$

n: number of lines cleared, h: maximum board height.

Approach

The Beginning: Our first approach was using the proposed architecture from the Deepmind Atari Paper on a high resolution standard tetris implementation. Due to the high resolution input and the high variance betweeen stones the learning was slow and didn't converge.

Initial Improvements: By downscaling the input to 1 block per pixel and removing unneeded color information, we could achieve much higher iteration rates. Further simplification of tetris to a 5x11 board with only two different stones (and) provided us with an environment with very good convergence rates to test different architectures. While this version of tetris is easier to play, we achieved comparable performance on the normal 10x22 tetris board with the same architectures.

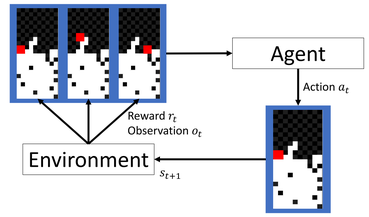
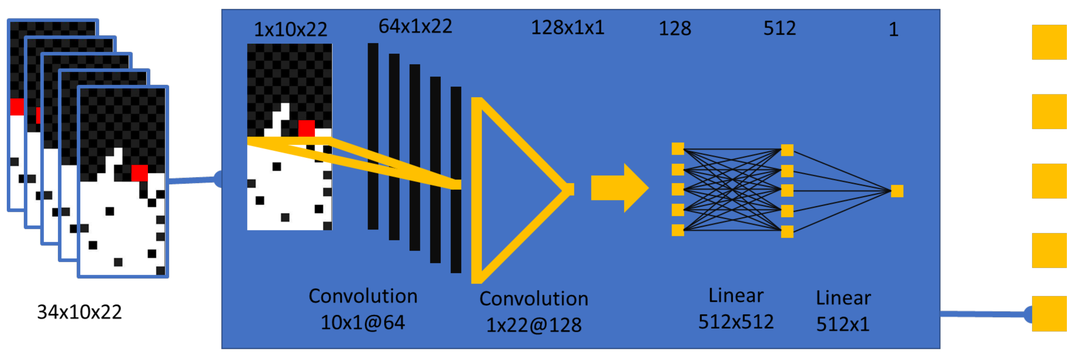


Figure 1: Setup of our Tetris-environment: The agent selects action a_t at time t, the environment computes reward r_t, observation o_t and new state $s_{e,t+1}$ with respect to a_t and $s_{e,t}$. In our case: $s_{a,t} = \emptyset$ and $s_{e,t} = o_t$ (the agent has no internal state and the agent observes the hole state space of the environment).

Problem of planning: Tetris is mainly a problem of planning stone placements. Because each placement can consist of up to 22 actions, this leads to very sparse rewards. To combat this issue, we used a modified tetris environment where each action is equal to one possible placement position. Different game states have different amounts of total possible actions, so we needed a custom architecture. Instead of learning normal Q-values $Q^*(s,a)$ for each possible action a from the state s_t, we instead directly learn the Q-value of the state $[s_{t+1}|a_t]$ under the assumption that we picked action a_t. While we need to evaluate each possible next state individually, the gains in faster convergence rates outweight the slower inference.

Network Architecture

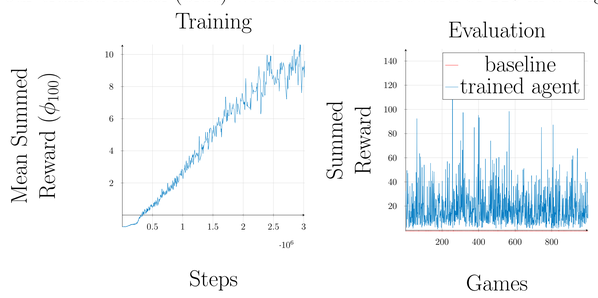


We employ a custom architecture using 1 dimensional convolutions to quickly reduce spatial size. After two additional linear layers we are left with the q-value of the proposed action. All layers except the last one use ReLU as the activation function. To address all possible tetromino placements, a batch of data consists of 34 possible placements[5] where the score is computed in parallel. In the end, the placement with the highest score is choosen and executed with regard to an ϵ-greedy policy.

[5]This represents the possible upper limit for different actions with a single stone on 10 wide board.

Results

In the left graph one can see a linearly increasing mean summed reward for the first 3 mio. training steps. We expect the curve to increase even further with a longer tarining phase. Unfortunately time was limited.
The right hand graph shows the evaluation of a randome agent (red) and our trained model (blue) with a maximum reward of 110 in a single game.



Conclusion

1. To benchmark our agent performance we compared it to a random playing agent, another implementation of two students from Stanford and a (near) Perfect Bot. Since the bot can play forever we compare the average number of lines deleted after 1000 placed tetrominos.

Name:	Our Agent	Random	Stanford	Perfect Bot
Lines per 1000 tetrominos	92	1	21	200

2. The network architecture from DQN was not really transferable. On the one hand we did not need to apply downconvolution to the input, since it was already small enough (22px × 10px) and on the other hand it seems like more computing time would not solve the problem completely. DeepMind trained for 50 mio steps (ca. 30 days).
3. Further work: As a next interesting step "blind Tetris" could be to approached. Therefore we would recommend using a Recurrent Neural Network (RNN). Also transfer learning would be an interesting topic, that could possibly lead to much faster learning success. The idea would be that you start with one tetromino and once the agent plays optimal you would add another tetromino and so on.

Other work and gimmicks:

- Implementation of a simple Human Tetris environment for a good comparison and in preparation for supervised learning.
- Approaches of supervised learning, where we prefilled the experience buffer with recorded games played in the simple Human environment.
- Using Tensorboard for statistcs and evaluation.
- Twitch Live Stream to follow the learning progress.

www.twitch.tv/koztantwo